

课堂小测

选择题

1. [单选题]

设计模式最早起源于哪个行业？

- A. 软件行业
- B. 建筑行业
- C. 金融行业
- D. 机器制造业

2. [单选题]

设计模式之父是谁？

- A. Kent Beck
- B. Erich Gamma
- C. Christopher Alexander
- D. GoF

3. [单选题]

关于设计模式的说法，哪个是正确？

- A. 设计模式是一段可以直接复制复用的代码模板。
- B. 设计模式是一种编程语言的语法规范
- C. 设计模式是软件项目的开发流程规范
- D. 设计模式是在特定环境下，为解决某一通用软件设计问题提供的一套定制解决方案，描述了对对象和类之间的相互作用。

画图题

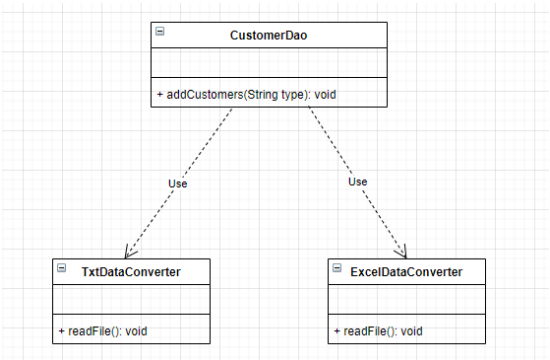
1.需求:现要开发一个 AI 翻译助手 App,支持文本语种检测、多语言互译等功能,其核心职责为完成用户输入文本的智能翻译及基础管理。 要求:定义一个翻译类,该类包含三个属性,分别为输入文本、翻译语言、翻译历史 还包含三个方法,分别为检测输入文本、执行翻译、保存结果到历史中。绘制 UML 类 图。

2. 需求:现要开发一个 AI 多模态识别系统,该系统支持文本、图像、语音三种类型的内容识别,其核心类包含:AI 基础模型类(AIModel)、文本识别模型类(TextRecognitionModel)、图像识别模型类(ImageRecognitionModel)、语音识别模型类(VoiceRecognitionModel)、

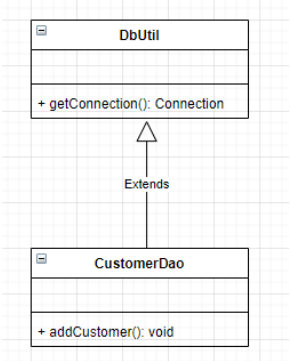
识别能力接口(IRecognizable)、识别引擎类(RecognitionEngine)。请分析它们之间的关系,并绘制出 JML 类图。

3. 需求:现要开发一个 AI 智能安防系统,该系统包含一些类,如 AI 安防系统烧烤(AISecuritySystem)、摄像头类(Camera)、人脸检测器类(FaceDDetector)报警记录类(AlarmRecord)。请分析它们之间的关系(关联、聚合、用户(User)、组合),并绘制出 UML 类图。

4. 请按照 OCP、LSP、DIP 进行重构



5. 请使用 CRP 进行重构



6. 某软件公司要开发一个加密算法库,在该加密算法库中提供了多种不同类型的加密算法,例如 DES (Data Encryption Standard, 数据加密标准) 加密算法、3DES (Triple DES, 三重 DES) 加密算法、MD5 (Message Digest Algorithm 5, 消息摘要算法 5) 算法、AES (Advanced Encryption Standard, 高级加密标准) 加密算法等。加密算法库开发人员希望设计一个较为灵活易用的加密算法库。在设计时,将每一种加密算法都封装到一个加密器类中,这些加密器类都继承自同一个父类(Encryptor)。由于加密器对象的创建过程较为复杂且经常被创建,因此需要封装加密器对象的创建过程。此外,为了方便调用,用户可以通过传入不同的参数来获得不同的加密器对象,例如传入参数“des”可得到一个 DES 加密器对象,传入参数“desede”可得到一个三重 DES 加密器对象,传入参数“md5”可得到一个 MD5 加密器对象。

要求: 请使用简单工厂模式来设计该加密算法库,画出 UML 图。

角色分析:

Encryptor (抽象产品)

DESEncryptor 、ThreeDESEncryptor、AEEncryptor 等（具体产品）
EncryptorFactory（工厂）

7. 某大学的校园卡系统需要管理学生的校园卡账户。在食堂、超市、图书馆等各个消费点，都需要对同一张校园卡进行扣款（deduce）和充值（recharge）操作。

要求：设计一个校园卡管理类（SchoolCardManager），确保整个系统中只有一个校园卡账户实例，绘制出 UML 类图。

8. 在软件开发中，界面组件通常需要适配多平台环境，不同平台的同类组件样式与行为存在差异。若直接在代码中实例化各类组件，会导致系统耦合度高、扩展困难，新增平台或组件时需大量修改原有代码，违反开闭原则。为实现跨平台组件的统一创建与管理，保证同一系列产品的兼容性，同时满足开闭原则，提升系统可维护性与扩展性，请使用抽象工厂模式设计一个跨平台 UI 组件生成程序，该程序支持 Window 和 Mac 平台，而且每个平台都能生成按钮（Button）和文本框（TextField）组件，请按要求绘制 UML 类图。

结构分析：

- 1.抽象产品：Button、TextField
- 2.具体产品：WindowButton、MacButton、WindowTextField、MacTextField
- 3.抽象工厂：UIFactory（包含 createButton、createTextField 两个方法）
- 4.具体工厂：WindowFactory、MacFactory

9. 现在的笔记本电脑只有 Type-C 接口，但大学生常用 3.5mm 圆孔耳机，接口不兼容无法使用。需要一个转接头（适配器），让圆孔耳机能插在 Type-C 电脑上使用。

角色分析：

- 1) 目标接口（Target）：TypeC（目标接口）
- 2) 适配者（Adaptee）：Headphone35mm（圆孔耳机）
- 3) 适配器（Adapter）：Adapter（转接头）
- 4) 客户端（Client）：Laptop（笔记本电脑）

请使用对象适配器进行设计，绘制 UML 类图。

10. 某互联网公司需要搭建组织架构管理系统，公司架构呈现典型树形层级结构：

- 1) 公司下设有多个职能部门（如研发部、人事部）；
- 2) 每个部门既可以包含员工，也可以下设子部门（如研发部下设后端组、前端组）；
- 3) 员工是最小单元；

系统需要实现统一功能：不管是单个员工，还是整个部门，都能统一执行信息展示，只有部门可以层级遍历、新增部门、新增员工等，无需改动原有核心代码，方便业务扩展。请根据上面要求，绘制出 UML 类图，拍照上传。

抽象构件：OrganizationComponent（组织机构抽象类）

叶子构件：Employee（员工）

容器构件：Department（部门）

11. 在一个 OA 办公系统中，员工可下发新建文档、删除文档、保存文档操作指令。指令统一交由办公助手调度执行，助手只调用命令，不实际操作文件，文件处理类真正完成文件业务，实现指令与执行解耦。请使用命令模式进行设计，绘制 UML 类图，完成后拍照上传。

分析：

办公助手（Assistant）-> 调用者

新建文档（CreateDoc）、删除文档（Assistant）、保存文档（SaveDoc）-> 命令

文件处理类（DocOperator）-> 接收者

12. 某图书管理系统需要开发一个图书查询功能，具体需求如下：

1.设计一个图书集合类，该类聚合了所有的图书；

2.设计一个迭代器类，用于迭代图书集合类中的所有图书；

3.迭代器类支持判断是否还有下一本图书、获取下一本图书的迭代功能；

请严格遵循迭代器模式规范：抽象聚合（AbstractBookList）、具体聚合（BookList）、抽象迭代器（AbstractBookIterator）、具体迭代器（BookIterator）四个核心角色，绘制 UML 类图。

13. 现开发一个电商库存预警系统，当商品库存低于安全阈值（10）时，自动通知仓库备货系统、运营后台系统。其中，商品库存（GoodsStock）作为目标（被观察者）；仓库（WarehouseObserver）和运营（OperateObserver）作为观察者。请使用观察者模式进行设计，绘制 UML 类图。

填空题

1./**

* 抽象产品：加密器抽象类

* 定义所有加密算法的统一接口

*/

public abstract class Encryptor {

// 抽象加密方法

public abstract String encrypt(String data);

}

```
/**
 * 具体产品：DES 加密器
 */
public class DESEncryptor extends Encryptor {
    @Override
    public String encrypt(String data) {
        return "DES 加密结果: " + data + "_DES";
    }
}
```

```
/**
 * 具体产品：3DES(Triple DES)加密器
 */
public class DESedeEncryptor extends Encryptor {
    @Override
    public String encrypt(String data) {
        return "3DES 加密结果: " + data + "_3DES";
    }
}
```

```
/**
 * 具体产品：MD5 加密器
 */
public class MD5Encryptor extends Encryptor {
    @Override
    public String encrypt(String data) {
        return "MD5 加密结果: " + data + "_MD5";
    }
}
```

```
/**
 * 工厂类：加密器工厂
 * 封装对象创建过程，根据参数返回对应加密器
 */
public class EncryptorFactory {
```

```
/**
 * 静态工厂方法：根据类型创建加密器
 * @param type 加密类型: des/desede/md5
 * @return 对应加密器对象
 */
public static Encryptor getEncryptor(String type) {
    // 统一封装复杂的创建逻辑
```

```

    Encryptor encryptor = null;
    switch (type.toLowerCase()) {
    case "des":
        encryptor = new DESEncryptor();
        break;
    case "desede":
        encryptor = new DESedeEncryptor();
        break;
    case "md5":
        encryptor = _____;
        break;
    default:
        throw new IllegalArgumentException("不支持的加密类型: " + type);
    }
    return encryptor;
}
}

```

```

/**
 * 客户端：使用加密算法库
 */
public class Client {
    public static void main(String[] args) {
        // 1. 获取 DES 加密器
        Encryptor des = _____;
        System.out.println(des.encrypt("测试数据"));
    }
}

```

2.

```

// 抽象工厂
public interface UIFactory {
    // 创建按钮
    Button createButton();
    // 创建文本框
    TextField createTextField()
}

```

```

// 按钮接口

```

```
public interface Button {  
    void display();  
}
```

```
// 文本框接口
```

```
public interface TextField {  
    void display();  
}
```

```
// Windows 按钮
```

```
public class WindowsButton implements Button {  
    @Override  
    public void ①_____ {  
        System.out.println("生成 Windows 风格按钮");  
    }  
}
```

```
// Mac 按钮
```

```
public class MacButton implements Button {  
    @Override  
    public void display() {  
        System.out.println("生成 Mac 风格按钮");  
    }  
}
```

```
// Windows 文本框
```

```
public class WindowsTextField implements TextField {  
    @Override  
    public void display() {  
        System.out.println("生成 Windows 风格文本框");  
    }  
}
```

```
// Mac 文本框
```

```
public class MacTextField implements TextField {  
    @Override  
    public void display() {  
        System.out.println("生成 Mac 风格文本框");  
    }  
}
```

```
// Windows 工厂
```

```
public class WindowsFactory implements ②_____ {  
    @Override
```

```
public Button createButton() {  
    return ③ _____;  
}
```

```
@Override  
public TextField createTextField() {  
    return new WindowsTextField();  
}  
}
```

```
// Mac 工厂  
public class MacFactory implements UIFactory {  
    @Override  
    public Button createButton() {  
        return new MacButton();  
    }  
}
```

```
@Override  
public TextField createTextField() {  
    return new MacTextField();  
}  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        // 使用 Windows 工厂  
        UIFactory windowsFactory = new WindowsFactory();  
        Button winBtn = windowsFactory.createButton();  
        TextField winField = windowsFactory.createTextField();  
        winBtn.display();  
        winField.display();  
        // 使用 Mac 工厂  
        UIFactory macFactory = ④ _____;  
        Button macBtn = macFactory.createButton();  
        TextField macField = _____;  
        macBtn.display();  
        macField.display();  
    }  
}
```


3.

// 目标接口

```
public interface TypeC {  
    void playByTypeC();  
}
```

// 适配者

```
public class Headphone35mm {  
    public void playBy35mm() {  
        System.out.println("播放音乐：李荣浩_爸爸妈妈~");  
    }  
}
```

// 适配器：转接头

```
public class Adapter implements ① _____ {  
    private Headphone35mm headphone;
```

```
    public Adapter(Headphone35mm headphone) {  
        this.headphone = headphone;  
    }
```

@Override

```
    public void playByTypeC() {  
        System.out.println("转接头工作：3.5mm → TypeC");  
        // 调用 headphone 对象的 playBy35mm 方法  
        ② _____  
    }  
}
```

// 客户端：笔记本电脑

```
public class Laptop {  
    public static void main(String[] args) {  
        // 1. 使用原生 TypeC 耳机  
        TypeC typeCHeadphone = new TypeC() {  
            @Override  
            public void playByTypeC() {  
                System.out.println("TypeC 耳机播放音乐：七里香~");  
            }  
        };  
        typeCHeadphone.playByTypeC();  
        System.out.println("-----");  
    }  
}
```

```

// 2. 使用圆孔耳机
Headphone35mm oldHeadphone = new Headphone35mm();
// 创建 Adapter 对象
TypeC adapter = ③ _____;
adapter.playByTypeC();
}
}

```

4.

// 抽象构件

```

public abstract class OrganizationComponent {
    private String name;

```

```

    public OrganizationComponent(String name) {
        this.name = name;
    }

```

// 添加

```

    public abstract void add(OrganizationComponent component);

```

// 删除

```

    public abstract void remove(OrganizationComponent component);

```

// 抽象方法：展示信息

```

    public abstract void showInfo();

```

```

    public String getName() {

```

```

        return name;

```

```

    }

```

```

}

```

// 叶子节点：普通员工

```

public class Employee extends ① _____ {

```

```

    private String job; // 岗位

```

```

    public Employee(String name, String job) {

```

```

        super(name);

```

```

        this.job = job;

```

```
}
```

```
@Override  
public void add(OrganizationComponent component) {  
    System.out.println("普通员工不能添加下属! ");  
}
```

```
@Override  
public void remove(OrganizationComponent component) {  
    System.out.println("普通员工没有下属可移除! ");  
}
```

```
@Override  
public void showInfo() {  
    System.out.println("员工: " + getName() + ", 岗位: " + job);  
}  
}
```

```
// 容器节点：部门  
public class Department extends OrganizationComponent {  
    // 存放员工、子部门  
    private List<OrganizationComponent> childList = new ArrayList<>();
```

```
    public Department(String name) {  
        super(name);  
    }
```

```
@Override  
public void add(OrganizationComponent component) {  
    // 往 childList 集合中添加 component  
    ② _____  
}
```

```
@Override  
public void remove(OrganizationComponent component) {  
    // 从 childList 集合中删除指定 component  
    ③ _____  
}
```

```
@Override  
public void showInfo() {  
    System.out.println("==== 部门: " + getName() + " =====");  
    // 递归遍历所有子节点  
    for (OrganizationComponent component : childList) {
```

```

        component.showInfo();
    }
}
}

public class Client {
    public static void main(String[] args) {
        // 1. 创建叶子员工
        OrganizationComponent emp1 = new Employee("张三", "UI 设计");
        OrganizationComponent emp2 = new Employee("李四", "软件测试");
        OrganizationComponent emp3 = new Employee("王五", "后端工程师");
        // 2. 创建容器部门
        OrganizationComponent devDept = new Department("研发部");
        OrganizationComponent backGroup = new Department("后端开发组");
        // 3. 组装树形结构
        devDept.add(emp1);
        devDept.add(emp2);
        backGroup.add(emp3);
        // 将 backGroup 添加到 devDept 中
        ④
        // 4. 调用 devDept 对象的 showInfo 方法，统一展示
        ⑤
    }
}
}

```

5.

```

// 1.抽象命令接口
interface Command {
    void ①____();
}

// 2.接收者：文件操作类
class DocOperator {
    public void createFile(){
        System.out.println("执行：新建办公文档");
    }
    public void deleteFile(){

```

```
System.out.println("执行：删除办公文档");
}
public void saveFile(){
System.out.println("执行：保存办公文档");
}
}
```

// 3.新建文档命令

```
class CreateDocCommand implements Command{
private DocOperator doc;
public CreateDocCommand(DocOperator doc){
this.doc = doc;
}
@Override
public void execute() {
doc.②_____.();
}
}
```

// 4.调用者：办公助手

```
class Assistant{
private Command cmd;
public void setCommand(Command cmd){
this.cmd = ③_____;
}
public void runTask(){
if(cmd != null){
cmd.④_____.();
}
}
}
```

// 测试主类

```
public class CommandTest {
public static void main(String[] args) {
DocOperator operator = new DocOperator();
Command createCmd = new ⑤_____;
Assistant assistant = new Assistant();
assistant.setCommand(createCmd);
assistant.runTask();
}
}
```

6.

// 图书实体类

```
class Book {  
    private String bookName;  
  
    public Book(String bookName) {  
        this.bookName = bookName;  
    }  
  
    public String getBookName() {  
        return bookName;  
    }  
}
```

// 1. 抽象迭代器接口

```
interface ①_____ {  
    // 判断是否有下一个元素  
    boolean hasNext();  
    // 获取下一个元素  
    Book next();  
}
```

// 2. 抽象聚合接口

```
interface ②_____ {  
    // 创建迭代器对象  
    AbstractBookIterator getIterator();  
}
```

```
abstract class AbstractBookList {  
    protected void addBook(Book book);
```

```
    protected AbstractBookIterator getIterator();  
}
```

// 具体聚合类：存储图书集合

```
class BookList extends AbstractBookList {  
    // 存储图书的数组  
    private List<Book> bookList = new ArrayList<>();
```

```
    // 添加图书
```

```
public void addBook(Book book) {  
    bookList.add(book);  
}
```

```
// 实现聚合接口的迭代器创建方法
```

```
@Override  
public AbstractBookIterator getIterator() {  
    return new BookIterator(③_____);  
}  
}
```

```
// 具体迭代器：书架迭代器（负责遍历书架图书）
```

```
class BookIterator implements AbstractBookIterator {  
    private BookList bookList;  
    private int index = 0;
```

```
public BookIterator(BookList bookList) {  
    this.bookList = bookList;  
}
```

```
// 判断是否还有下一本图书
```

```
@Override  
public boolean hasNext() {  
    // 核心边界判断：索引小于图书总数则存在下一个元素  
    return ④_____;  
}
```

```
// 获取下一本图书
```

```
@Override  
public Book next() {  
    // 返回当前元素并索引自增  
    return ⑤_____;  
}  
}
```

```
// 测试客户端
```

```
public class Client {  
    public static void main(String[] args) {  
        AbstractBookList bookList = new BookList();  
        bookList.addBook(new Book("Java 编程思想"));  
        bookList.addBook(new Book("数据结构与算法"));  
        bookList.addBook(new Book("设计模式详解"));
```

```
    // 获取迭代器遍历图书
```

```

AbstractBookIterator iterator = bookList.getIterator();
while (iterator.hasNext()) {
    Book book = iterator.next();
    System.out.println("遍历图书: " + book.getBookName());
}
}
}

```

7.

//抽象被观察者

```

public abstract class Subject {
    //保存所有订阅观察者
    protected List<Observer> obs = new ArrayList<>();

```

//添加观察者

```

    public void addObserver(Observer o){
        obs.add(o);
    }

```

//移除观察者

```

    public void removeObserver(Observer o){
        obs.remove(o);
    }

```

//通知所有观察者

```

    public abstract void notifyAllObserver(____①____);
}

```

//抽象观察者

```

public interface Observer {
    //接收被观察者推送的库存数据
    void ____②____;
}

```

// 具体被观察者

```

public class GoodsStock extends Subject{
    private int stock;

```

```

    public void setStock(int stock){
        this.stock = stock;

```

//库存低于 10 触发通知

```

        if(stock < 10){
            ____③____;

```



```
}  
}
```

```
@Override  
public void notifyAllObserver(int stockNum) {  
    //遍历所有观察者推送消息  
    for(Observer ob : ____④____){  
        ob.update(stockNum);  
    }  
}  
}
```

```
// 具体观察者（仓库）  
public class WarehouseObserver implements Observer{  
    @Override  
    public void update(int stockNum) {  
        System.out.println("库存预警，当前剩余: "+stockNum);  
    }  
}
```

```
// 具体观察者（运营后台）  
public class OperateObserver implements Observer{  
    @Override  
    public void update(int stockNum) {  
        System.out.println("库存预警，当前剩余: "+stockNum);  
    }  
}
```

```
// 客户端  
public class Test {  
    public static void main(String[] args) {  
        //1.创建被观察者商品库存  
        GoodsStock goods = new GoodsStock();  
        //2.创建观察者  
        Observer warehouse = new WarehouseObserver();  
        Observer operate = new OperateObserver();  
        //3.订阅注册  
        goods.addObserver(warehouse);  
        goods.addObserver(operate);  
        //4.修改库存触发预警  
        ____⑤____;  
    }  
}
```